Read Deep, Write Shallow: How Looped Language Models Should Spend Their Depth

Yang Hu

Abstract

Looped language models reuse one block of layers several times per token and, under fixed-depth inference, spend the same number of loops on the prompt (prefill) and on every generated token (decoding). On the checkpoints and benchmarks we test, the two buy different things: *prefill depth mainly buys knowledge and long-context accuracy, while decoding depth mainly buys multi-step reasoning*, with exceptions that depend on the prompting protocol. A small connector trained on a frozen looped model separates the two budgets: one set of parameters serves all 16 pairs of prefill and decoding depth up to four loops, and one stored prompt KV cache serves every decoding loop. On Ouro-2.6B-Thinking, reading the prompt with four loops and writing with two scores at least as high as the native two-loop model on all eight standard benchmarks we test (significantly higher on five), at the same decoding depth, and within two points of the native four-loop model on six; on GSM8K it is 1.0 point below the four-loop model on the full test set and, on 64 timed problems, answers in $0.55\times$ its time. The same split, with matched continued training, makes looped decision models more accurate when they decide with one loop. Code, results and per-item outputs: `https://github.com/yanghu819/read-deep-write-shallow`.

1 Introduction

Looped language models such as Ouro (Zhu et al., 2025), Huginn (Geiping et al., 2025) and Mixture-of-Recursions (Bae et al., 2025) obtain depth without adding parameters by running a shared block of layers several times per token (Dehghani et al., 2018). Under the fixed-depth inference we study, the price is paid twice: every generated token runs every loop, and every loop keeps its own key–value (KV) cache.

Yet the two phases of inference are not alike. Reading the prompt (prefill) is parallel and paid once per request; writing the answer (decoding) is sequential and paid for every token. A looped model nevertheless spends the same depth on both. What does depth buy when reading, and what does it buy when writing? If the answers differ, a model could read deep and write shallow.

We write read p / write d for prefilling the prompt with p loops and generating every answer token with d loops, and native d/d for the unmodified model, which uses d loops for both. Answering the question needs a model that writes with fewer loops than it read with, so its shallow writing loops must attend to KV produced by a deeper loop. Our *depth-split connector* combines LoRA on the frozen model with a learned map on the prompt KV for every writing loop (Figure 2); one stored prompt cache serves every writing loop. We train one connector and evaluate it at all 16 pairs of read and write depth with its parameters fixed.

The resulting map shows a division of labour (Figure 1). On the benchmarks we test, reading depth mainly moves knowledge and long-context accuracy, and writing depth mainly moves multi-step reasoning. With the writer held fixed, reading deeper helps on five checkpoints from two model families (Table 1).

Reading deep and writing shallow puts the two findings together (Figure 4): on Ouro-2.6B-Thinking, reading four loops and writing two scores at least as high as native 2/2 on all eight benchmarks at the same decode depth (significantly higher on five), and within two points of native 4/4 on six. On full GSM8K it scores 93.3 against 94.2 for native 4/4; on 64 timed problems its answer time, prefill included, is $0.55\times$ that of native 4/4 (Figure 7). Against measured decode throughput, the connector is more accurate than native models at one and two writing loops and faster at three and four (Figure 5). The split carries over to looped decision models: against a control continued with the same data and updates, Lev

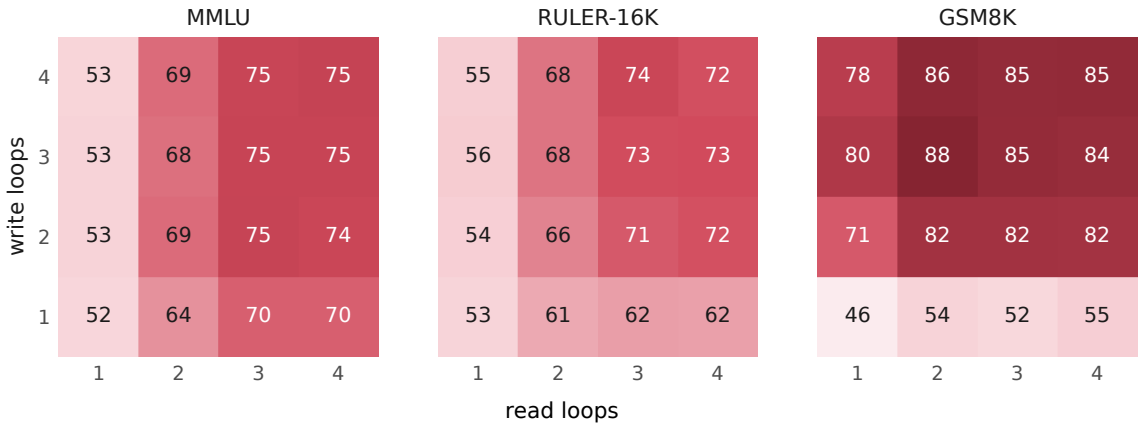


Figure 1: **Reading deeper improves knowledge and long-context accuracy; writing deeper improves multi-step reasoning.** One connector on frozen Ouro-2.6B evaluated at all 16 (read, write) pairs. Knowledge (MMLU) and long-context reading (RULER-16K) improve along the read axis; multi-step reasoning (GSM8K) improves along the write axis. Grid subsets: MMLU 20 questions per subject, RULER-16K 260 rows, GSM8K 400 problems; cells show accuracy (%), one colour scale for all panels; full-set comparisons are in Table 2.

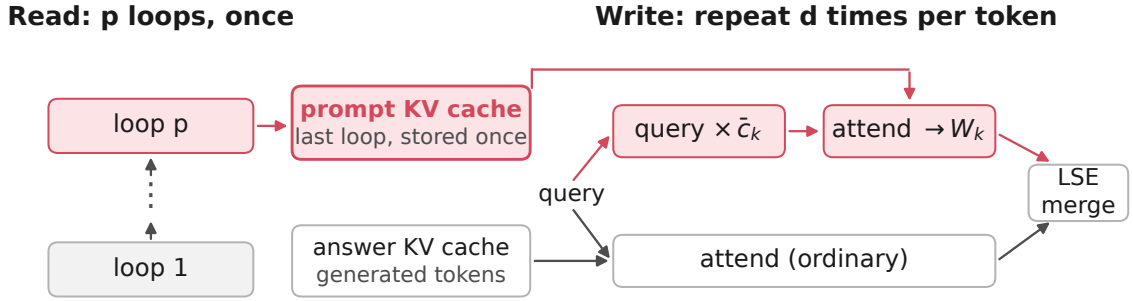


Figure 2: **The depth-split connector.** The frozen model reads the prompt with p loops and keeps one prompt KV cache. Each of the d writing loops reads that cache through its own map: a learned scale on each rotary (RoPE) pair of the keys, which can be moved onto the query so the cache is never rewritten, and a value map applied after attention. Attention over the prompt and over the generated tokens is computed separately and combined into one softmax, as in blockwise (FlashAttention-style) attention. The prompt KV is stored once for every d: total KV $b(n + d\,t)$ instead of $b\,d(n + t)$, with b bytes per token and loop and n, t the prompt and answer length.

models that read their context deep are 4.4–7.5 points more accurate at one decision loop on three backbones (Figure 6).

Our contributions are a measurement of what reading and writing depth each buy in looped language models, a connector that makes the split usable on existing checkpoints with an exact single prompt cache, and an end-to-end measurement of the time it saves. Code, per-item results and a script that recomputes the paper's statistics accompany it.

2 Related Work

Looped and elastic depth. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) train one model to serve several depth budgets; ELR also reports that keeping only the last loop's KV at inference retains most of the gain. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) let later loops read the first loop's global KV plus a short window of their own. All of them spend the same depth on prompt and answer.

Phase-specific computation. Continuous Depth Batching (Schwethelm et al., 2026) runs full-depth prefill and exits early during decode; Position-Aware Depth Decay (Fan et al., 2025) skips more layers for later tokens; OverFill (Kim et al., 2025) prefills with the full model and decodes with a width-pruned one. Fixed Points Part II (Huang et al., 2026) distils a lightweight prefill network, followed by one teacher recur-

rence, while keeping the teacher's decoding depth; POP (He et al., 2026) and SPEED (Oh et al., 2026) use shallower prefill than decode. Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and Almost Free State-Prediction Separation (Langford et al., 2026) add decode-side computation. These works change decode depth, reduce prefill computation or share caches across recurrences; we instead vary prompt and answer depth independently within one trained connector and measure what each buys.

Asymmetric KV producers and consumers. YOCO (Sun et al., 2024) splits a decoder into a self-decoder that produces one shared global KV cache and a cross-decoder that reads it; YOCO-U (Sun et al., 2026) loops the self-decoder with shared parameters, keeping global KV storage independent of the number of loops while every token still runs the loops; KITE (Hu et al., 2026) expands a checkpoint into two towers in which bulk prefill runs only the KV-producing tower. These architectures are designed or expanded for an asymmetric split. We instead adapt an existing looped model after pretraining and choose prompt depth p and answer depth d independently per request: the frozen backbone produces the prompt KV once with p loops, and generation uses d loops through a small connector.

Sharing and transferring KV. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops; BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; Cache-to-Cache (Fu et al., 2025), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models; BellowsKV (Zhang and Chen, 2026) shares one latent across layers. As in CacheLink's output-supervised corrections, our maps are trained through the receiver's output distribution. Within one Ouro model, tokens, layers and positions align, so a RoPE-commuting scale and a value map suffice, and serving them from one stored cache is exact.

3 Method

We describe the connector for Ouro; SenseNova-Loop-9B uses a lighter variant without maps (Appendix A).

Reading and writing. "read p / write d" prefills the prompt with p loops and decodes every answer token with d loops; native d/d is the unmodified model. The frozen model reads the prompt with p loops and keeps only the last loop's prompt keys and values. The writer is the same frozen model with rank-16 LoRA (Hu et al., 2021) on its attention projections, running d loops per token.

RoPE-commuting maps. Writing loop k reads the prompt KV cache through a map per (loop, layer, KV head). The key map is one complex scale c_j per rotary frequency pair j (Su et al., 2021): viewing the pair of key coordinates (k_{2j}, k_{2j+1}) as a 2-D vector, $c_j = r\,e^{i\phi}$ stretches it by r and turns it by ϕ. Writing a rotated key pair as a complex number $\tilde{k}_j = e^{im\theta_j} k_j$, scaling commutes with the rotation, $c_j \tilde{k}_j = e^{im\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j \tilde{k}_j\rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j\rangle$: the map is applied to the query and the stored cache is never rewritten. The value map is a full linear map W_k per head; because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$ is applied once to the attention output over the prompt. Prompt and generated tokens share one softmax: the two parts are computed by two attention calls with outputs o_P, o_A and log-sum-exps ℓ_P, ℓ_A of their attention logits, and merged as $g\,o_P + (1-g)\,o_A$ with $g = \sigma(\ell_P - \ell_A)$, which equals one softmax over both (the blockwise softmax of FlashAttention (Dao, 2023)). The prompt KV is therefore stored once for every d, and total KV is $b(n + d\,t)$ instead of $b\,d(n+t)$. Serving from one cache equals materialising a mapped prompt cache per loop; in bfloat16 all 512 checked greedy tokens agree.

What the maps do. With the same training and the maps frozen at the identity, the writer loses little on short-context reasoning and knowledge but 11.4 points on RULER-16K at read 4 / write 1 (Appendix F): the maps matter mainly for long prompts.

Training. We minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over answer positions, with the native 4/4 model as teacher, on 48M answer tokens of reasoning traces, instruction data, long-context data and web text. One (p, d) pair is sampled per micro-batch, so one set of parameters (63M, 2.4% of Ouro-2.6B) serves all 16 pairs. Details are in Appendix A.

```
# blocks: frozen looped block, L layers; p/d: read/write loops
# c[k][l] = 1+a+1j*b per RoPE pair; W[k][l] = I+delta (init: I)
# LoRA on q, k, v, o is active only while writing.

def read(prompt, p): # prompt minus its last token
    h = embed(prompt)
    for _ in range(p):
        for l in range(L):
            h, K[l], V[l] = blocks[l](h) # last loop's K, V kept
    return K, V # one prompt KV cache

def attend(q, K, V, kv_answer, c, W): # layer l, writing loop k
    qc = complex_mul(q, c.conj()) # key scale moved to the query
    o_p, lse_p = flash_attn(qc, K, V) # prompt part
    o_p = o_p @ W.T # attention is linear in V
    o_a, lse_a = flash_attn(q, *kv_answer, causal=True)
    g = torch.sigmoid(lse_p - lse_a) # exact softmax over both
    return g * o_p + (1 - g) * o_a

def write(token, K, V, d): # last prompt token, then answer
    h = embed(token)
    for k in range(d):
        for l in range(L):
            h = blocks[l](h, attn=lambda q, kv: attend(
                    q, K[l], V[l], kv, c[k][l], W[k][l]))
    return lm_head(h)
```

```
# training: one (p, d) per micro-batch,
# teacher = the native 4/4 model
p, d = randint(1, 4), randint(1, 4)
with torch.no_grad(): t = model(x, read=4, write=4)
s = model(x, read=p, write=d) # LoRA + maps
kl = kl_div(s.log_softmax(-1),
          t.log_softmax(-1), log_target=True,
          reduction="none").sum(-1)
loss = kl[answer].sum() / answer.sum()
```

Figure 3: **The depth-split connector in PyTorch-style pseudocode.** The prompt KV cache is written once by the last of p read loops; every writing loop reads it through its own key scale (applied to the query) and value map (applied to the attention output), and the prompt and answer attention are combined exactly through their log-sum-exp.

4 Experiments

Setup. Models: Ouro-2.6B, Ouro-1.4B and their Thinking variants (Zhu et al., 2025) (four loops), Nanbeige4.2-3B-Base (two loops) and SenseNova-Loop-9B-Preview, a hybrid of Gated-DeltaNet and full-attention layers whose middle block is visited twice. Benchmarks: GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2023), MMLU (Hendrycks et al., 2020), MMLU-Pro (Wang et al., 2024), ARC-Challenge (Clark et al., 2018), BBH (Suzgun et al., 2022), RULER (Hsieh et al., 2024) at 4K/8K/16K and HELMET (Yen et al., 2024); protocols and set sizes are in Appendix B. Every comparison is paired over items with 95% bootstrap intervals. To isolate reading depth from adaptation we compare read p / write d with read d / write d of the *same* connector; comparisons with native models include the effect of training the connector.

4.1 What reading depth and writing depth buy

Figure 1 shows the full grid on Ouro-2.6B. MMLU rises from 52–53 at one read loop to 70 at three or four with a one-loop writer and to 74–75 with deeper writers; GSM8K gains 9 points along the read axis at one write loop but rises from 46–55 to 78–88 with writing depth. Table 2 confirms the pattern on full sets: at one writing loop, reading four loops instead of one adds 15.2 MMLU, 8.6 RULER-16K and 8.0 GSM8K points; the gain shrinks as writing deepens. A second training seed reproduces these numbers (Appendix C).

The same holds across models (Table 1). With the writer fixed at one loop, reading deeper improves knowledge and long-context tasks and the selected multi-step task on all five checkpoints; Ouro-1.4B is the exception on long context (RULER-16K $+1.4$, not significant). The gains are task-dependent: on both 1.4B checkpoints the same change sharply lowers 3-shot BBH (Limitations). On Nanbeige4.2-3B-Base separately trained connectors per (p, d) pair point the same way but do not isolate reading depth (Appendix B).

Reading deep is not uniformly better. At one writing loop on Ouro-2.6B it lowers 4-shot MATH-500 by 11.2 points while raising zero-shot MATH-500 by 7.0; it sharply lowers 3-shot BBH on both 1.4B checkpoints (-24.5 and -20.5); and on Ouro it lowers exact key–value recall in HELMET (Appendix H). The division of labour is a tendency on the benchmarks we test, and it depends on how the task is prompted.

model (same writer, write 1)	knowledge	long context	multi-step
Ouro-2.6B, read 4 vs. 1	MMLU +15.2 [14.3, 16.1]	RULER-16K +8.6 [4.3, 12.8]	GSM8K +8.0 [5.3, 10.5]
Ouro-1.4B, read 4 vs. 1	ARC-C +10.8 [8.4, 13.1]	RULER-16K +1.4 [−2.6, 5.4]	GSM8K +6.2 [3.6, 8.7]
Ouro-2.6B-Thinking, read 4 vs. 1	MMLU-Pro +18.6 [13.6, 23.8]	HELMET +7.7 [2.1, 13.6]	GSM8K +6.6 [4.2, 9.2]
Ouro-1.4B-Thinking, read 4 vs. 1	MMLU-Pro +17.1 [12.4, 22.4]	HELMET +4.6 [0.1, 8.6]	GSM8K +13.6 [10.5, 16.6]
SenseNova-Loop-9B, read 2 vs. 1	MMLU-Pro +5.5 [2.9, 8.1]	HELMET +3.5 [0.8, 6.1]	BBH +7.0 [5.9, 8.0]

Table 1: **Reading depth with the writer held fixed** across five checkpoints (paired, within one connector). Set sizes: MMLU 14,042, ARC-C 1,172, MMLU-Pro 420 (Ouro) and 980 (SenseNova), RULER 520, HELMET 1,108 (Ouro) and 6,646 (SenseNova), GSM8K 1,319, BBH 6,511. HELMET is averaged as its authors do (mean per dataset, then per category, then over its four categories). Each column shows one benchmark of that kind per checkpoint; counterexamples are discussed in Section 4.1 and Appendix H. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

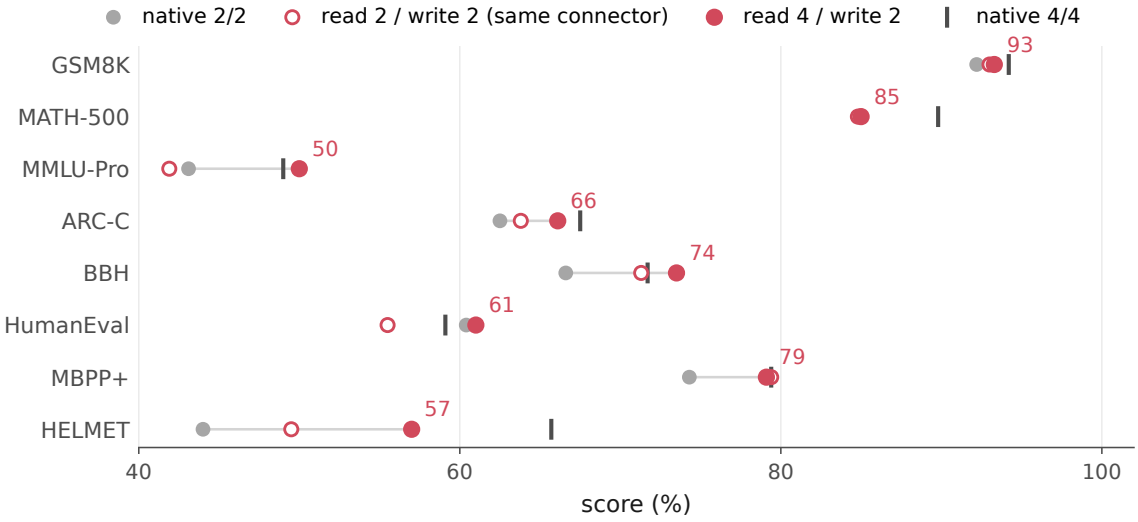


Figure 4: **Read deep, write shallow: the decode depth of two loops, close to the quality of four.** Ouro-2.6B-Thinking on standard benchmarks (GSM8K, MATH-500, MMLU-Pro, ARC-Challenge, BBH, HumanEval, MBPP+, and HELMET averaged over its categories); read 4 / write 2 uses one connector and one stored prompt KV cache. Hollow circles: the same connector reading only two loops, which separates the effect of training from that of reading depth. All models are scored on identical evaluation sets.

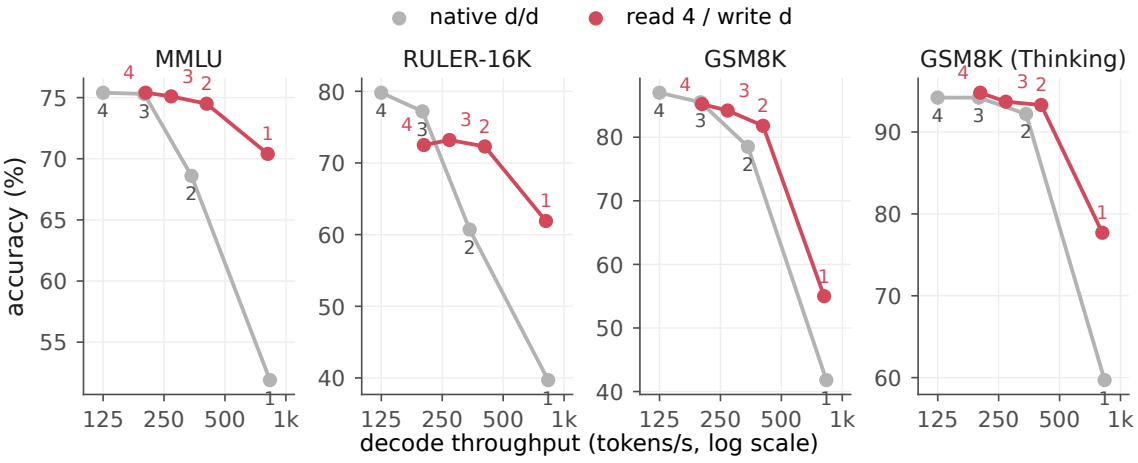


Figure 5: **Accuracy against decode throughput.** Native d/d (grey) and the connector reading 4 loops and writing d (red), $d = 1$–4 (labels). Throughput: best decode tokens/s over batch sizes at 8K context on one B300 (same for the base and Thinking models, which share the architecture); one stored prompt KV cache admits larger batches as d grows. Ouro-2.6B on the grid subsets of Figure 1; Ouro-2.6B-Thinking on all 1,319 GSM8K problems.

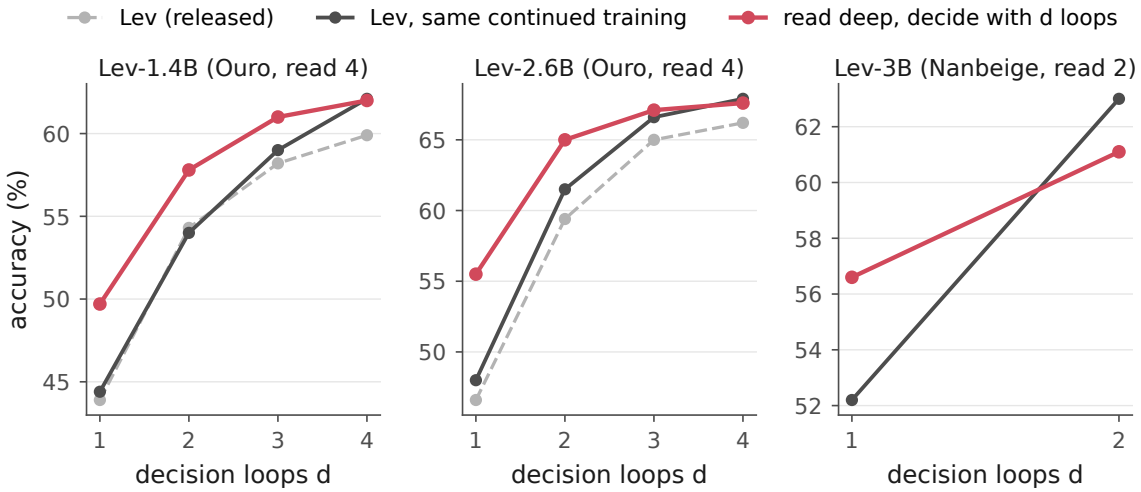


Figure 6: **Read deep, decide shallow.** Lev looped decision models (Song et al., 2026) on the public LevBench development set (3,270 answerable questions): accuracy against the number of loops spent on the decision. Red: the context is read deep (four loops on Ouro, two on Nanbeige) and the question and options run d loops on that deep KV; dark grey: the same continued training with Lev's own forward; dashed: the released model.

write d	MMLU	RULER-16K	GSM8K
1	$+15.2$ [14.3, 16.1]	$+8.6$ [4.3, 12.8]	$+8.0$ [5.3, 10.5]
2	$+6.0$ [5.4, 6.7]	$+5.3$ [1.3, 9.4]	-1.7 [-3.6, 0.2]
3	$+0.6$ [0.2, 1.1]	-0.4 [-3.0, 2.2]	-0.8 [-2.4, 0.8]

Table 2: **Reading depth at fixed writing depth** (Ouro-2.6B, one connector, full sets): read 4 / write d minus read d / write d. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

4.2 Read deep, write shallow

Figure 4 puts the two findings together on Ouro-2.6B-Thinking: read four loops, write two. At the decode depth of native 2/2, read 4 / write 2 scores higher on all eight benchmarks, significantly on MMLU-Pro ($+6.9$ [3.1, 11.0]), BBH ($+6.9$ [4.6, 9.2]), ARC-Challenge ($+3.6$ [2.0, 5.3]), MBPP+ ($+4.8$ [1.3, 8.2]) and HELMET ($+13.0$ [6.7, 19.5]); on GSM8K, MATH-500 ($+0.2$ [-2.6, 3.0]) and HumanEval the gains are within noise. Against native 4/4, which writes with twice the depth, it is within two points on six of the eight benchmarks and trails on HELMET (-8.7), where native 4/4 recalls exact key–value pairs that deep reading loses (Appendix H), and on MATH-500 (-4.8 [-7.8, -2.0]).

Part of the gain over native 2/2 comes from training the connector, not from depth. The same connector reading two loops (hollow circles; same weights, same training, same writing depth) isolates reading depth: reading four rather than two loops adds $+8.1$ [4.0, 12.1] on MMLU-Pro, $+7.5$ [1.4, 13.8] on HELMET, $+2.3$ [0.9, 3.7] on ARC-Challenge and $+2.2$ [0.1, 4.3] on BBH, and shows no detectable difference on GSM8K ($+0.2$), MATH-500 ($+0.2$) and MBPP+ (-0.3). On MMLU-Pro the gain over native 2/2 is accounted for by reading depth; on MBPP+, by training. On Ouro-1.4B-Thinking the effect of reading depth repeats on the same eight benchmarks, but that connector loses 3-shot BBH against native 2/2 (Appendix D).

Figure 5 shows the same trade-off across all writing depths, against measured decode throughput. For every writing depth, reading four loops moves the model up (more accurate at one or two writing loops: MMLU $+18.4$ and $+5.9$, RULER-16K $+22.2$ and $+11.6$ on the grid subsets) or right (faster at three or four writing loops, because one stored prompt cache admits larger batches: 271 and 203 against 200 and 125 tokens/s). Read 4 / write 2 runs at $1.19\times$ the throughput of native 2/2 and $3.2\times$ that of native 4/4.

4.3 Writing shallow is fast end to end

We time whole answers at batch 1, prefill included (Figure 7; Table 5 in Appendix E). On Ouro-2.6B-Thinking a connector that writes with two loops answers GSM8K in $0.55\times$ [0.45, 0.67] the time of native 4/4. On all 1,319 problems, read 4 / write 2 scores 93.3 against 94.2, a paired difference of -1.0 [-2.0, 0.1]. Reading 4 rather than 2 loops before writing two changes the answer

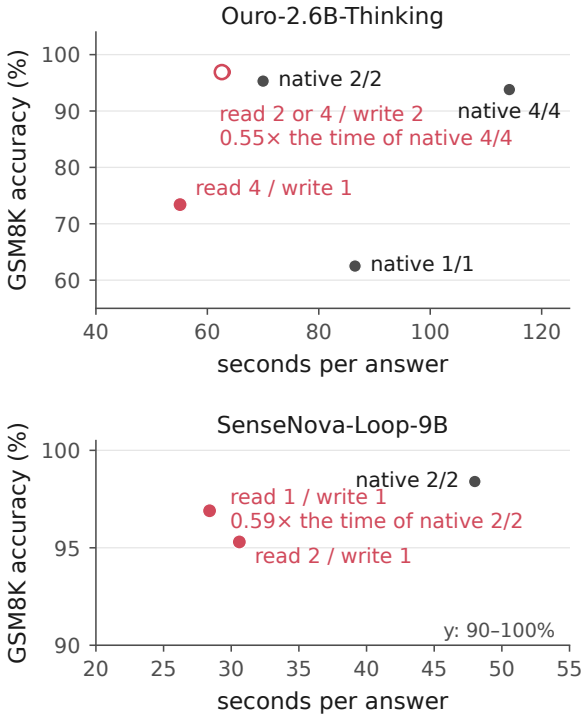


Figure 7: **Writing shallow saves time end to end.** Accuracy vs. seconds per answer at batch 1 (prefill included, real greedy answers, the same 64 GSM8K problems for every point). Top: writing with two loops takes $0.55\times$ the time of native 4/4, whether the prompt is read with 2 or 4 loops; on all 1,319 problems read 4 / write 2 is 1.0 point below native 4/4 (Section 4.3). Bottom: on SenseNova-Loop-9B, whose native one-loop model solves 2.2% of all GSM8K problems, read 1 / write 1 is 1.6 points below native 2/2 at $0.59\times$ the time (y axis zoomed).

time by $1.01\times$ [0.90, 1.11]: the saving comes from the shallower, adapted writer, including its shorter answers. On the 40 timed MATH-500 problems read 4 / write 2 scores 85.0, against 82.5 for native 4/4 and 85.0 for native 2/2, at $0.63\times$ the time of native 4/4 but $1.22\times$ the time of native 2/2, because its solutions are not shorter. On SenseNova-Loop-9B the trained read 1 / write 1 scores 95.6 against 95.8 for native 2/2 on all 1,319 GSM8K problems and takes $0.59\times$ [0.52, 0.68] the time per answer, close to the ratio of layer executions per token (32 vs. 56).

4.4 Decision models

The split also serves looped decision models. Lev (Song et al., 2026) is a looped model that, given a context and a question with options, assigns probabilities to the options with a shared learned scoring head after every loop, without generating text. We read the context with four loops (two for Lev-3B, whose backbone has two) and let every decision loop attend to the last read loop's KV; we add no maps and continue training Lev's own LoRA and scoring head. The control continues the same released weights on the same data with Lev's own forward: both arms take 140 updates on the 8,925-record public S31 training subset with Lev's hyperparameters. Our reference forward runs the full sequence through both the read and the decision loops, so training compute is not matched (we check this below). We evaluate on the 3,270 answerable questions of LevBench's public development set and never touch its locked test set; intervals resample record groups, as Lev does.

With one decision loop, reading deep raises accuracy by $+5.3$ [3.4, 7.2] on Lev-1.4B, $+7.5$ [5.7, 9.5] on Lev-2.6B and $+4.4$ [2.8, 6.1] on Lev-3B (Figure 6). The advantage shrinks as the decision deepens: at the full depth the Ouro intervals include zero, and Lev-3B is 1.8 points worse. The control itself moves little at one loop ($+0.5$ on Lev-1.4B, $+1.3$ on Lev-2.6B against the released models), and without training, reading deep does not help (-3.0 on Lev-1.4B). The result holds across training seeds: repeating both arms with other seeds, the one-loop gain ranges from $+3.9$ to $+5.3$ on Lev-1.4B and from $+5.1$ to $+7.5$ on Lev-2.6B, with every interval excluding zero (four seeds each); on Lev-3B it is $+4.4$, $+5.1$, $+4.0$ and $+0.9$ [-0.8, 2.7] (mean $+3.6$), so one of its four seeds shows no clear gain. The gain persists against a control trained for two epochs (twice the updates, about the training compute of the read-deep arm), which still trails reading deep at one loop by $+4.0$ [2.1, 6.0] on Lev-1.4B, $+5.5$ [3.7, 7.4] on Lev-2.6B and $+4.0$ [2.3, 5.7] on Lev-3B.

5 Conclusion

Depth in a looped language model is two budgets, not one. On the checkpoints and benchmarks we test, reading depth is paid once and mainly buys knowledge and long-context accuracy, while writing depth is paid per token and mainly buys multi-step reasoning; both effects depend on the task. A small connector makes the two separable on existing checkpoints with one exact prompt cache, and a shallower adapted writer cuts the measured time per answer at a small accuracy cost.

Limitations

Reading deep lowers 4-shot MATH-500, gives a negative but inconclusive estimate on RULER-4K and, on Ouro, lowers exact key–value recall in HELMET (Tables 9 and 8); at three or more writing loops, one deep prompt KV cache trails the native model on RULER ($-10.4/-9.5/-5.3$ at 4K/8K/16K, write 3). Deeper prefill raises time to first token, so it pays when answers are long or quality requires it. Both 1.4B checkpoints degenerate at read 4 / write 1 on 3-shot BBH (Ouro-1.4B: -24.5, Ouro-1.4B-Thinking: -20.5 against read 1 / write 1); with two writing loops, reading four instead of two helps (Ouro-1.4B-Thinking: $+8.8$, Table 4). On Nanbeige4.2-3B-Base our shared-connector recipes trail connectors trained per (p, d) pair, so Appendix B reports per-pair results. Gains against native models include the adaptation of the connector. Throughput is a controlled decode benchmark and end-to-end time is measured at batch 1 on one GPU type. Thinking checkpoints on raw HELMET prompts can stop at once; we decode them so that an answer cannot be empty, for every configuration (Appendix B). Probes measure linear decodability, not causal use. All models are 1.4B–9B, decoding is greedy and benchmarks are English.

References

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2018. Universal Transformers. *arXiv preprint arXiv:1807.03819*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. *arXiv preprint arXiv:2503.08524*.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2025. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. *arXiv preprint arXiv:2510.03215*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. *arXiv preprint arXiv:2602.03295*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Zhiheng Hu, Yixun Wei, Jian Zhou, Yizhuang Zhou, Ji Li, Xing Chen, Yang Li, Bojun Wang, Yibo Zhu, Xiangyu Zhang, and Daxin Jiang. 2026. KITE: KV-Invariant Transformer Expansion for Efficient Agentic LLM Scaling. *arXiv preprint arXiv:2609.27294*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Aur Shalev Merin. 2026. Present but Hard to Read: Probing Latent Reasoning with Minimal Pairs. *arXiv preprint arXiv:2610.28498*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Dingjie Song, Zhiling Yan, Yuxuan Zhang, and Lichao Sun. 2026. LEV: Thinking Fast and Deep with Looped Decision Models. *arXiv preprint arXiv:2610.20951*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*.

Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. 2026. Universal YOCO for Efficient Depth Scaling. *arXiv preprint arXiv:2604.01220*.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. *arXiv preprint arXiv:2405.05254*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261*.

Elena Voita, Rico Sennrich, and Ivan Titov. 2019. The Bottom-up Evolution of Representations in the Transformer: A Study with Machine Translation and Language Modeling Objectives. In *Proceedings of EMNLP-IJCNLP 2019*, pages 4396–4406.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv preprint arXiv:2406.01574*.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. 2023. Looped Transformers are Better at Learning Learning Algorithms. *arXiv preprint arXiv:2311.12424*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2024. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. *arXiv preprint arXiv:2410.02694*.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

A Training Details

Connector. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only when writing; per (writing loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B. The backbone is frozen.

Objective and data. $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ summed over answer positions and normalised by the answer tokens in the global batch; teacher = native 4/4 with the same weights; one pair (p, d) sampled per micro-batch. 48M answer tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of answer tokens); prompts up to 7,680 tokens, answers up to 1,024. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data was deliberately included; we did not decontaminate the training corpora.

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 B300 GPUs (6.75 hours for Ouro-2.6B), bfloat16 backbone.

Answer stored once. Generated tokens keep one KV cache per layer shared by all writing loops; at inference the last writing loop commits each token, and earlier loops read that history plus the current token's KV computed live in their own loop (*self-live*[1], a layout also used by LSTM-style looped transformers (Kirsch, 2026)). For example, when loop 1 processes token 5, it attends to tokens 1–4 through the last loop's stored KV and to token 5 through its own, freshly computed KV. Training uses the same KL objective in this layout, in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt KV cache, that recorded history strictly before the current position, and the current token's KV computed live on the diagonal. Training thus approximates, but does not exactly reproduce, inference, where the history is generated recursively in the shared cache. With prompt and answer each stored once, total KV is $b(n + t)$, that of a one-loop model; with 1K-token prompts and 2,048 generated tokens the largest batch that fits grows from 32 (native 4/4) to 192 and write-4 decode throughput from 219 to 476 tokens/s (759 vs. 586 at d=2), at a cost of 4.8 GSM8K points against native 4/4 (Table 7 compares it with per-loop answer caches). On Ouro-2.6B-Thinking, a connector trained the same way changes full-set GSM8K at read 4 / write 2 by $+0.5$ [−0.5, 1.6] against per-loop answer caches; at batch 1 its cost per generated token is the same (82.5 vs. 83.1 ms) but its answers are longer (1,166 vs. 741 tokens), so it takes $1.53\times$ [1.27, 1.84] the time per answer. Its benefit is memory and batch size, not batch-1 latency.

SenseNova-Loop-9B. Rank-16 LoRA on the attention projections of the looped layers (full attention: q/k/v/o; Gated-DeltaNet: input and output projections), active only when writing; no maps. The one-visit writer reads the prefill state of the last of $p \in \{1, 2\}$ visits (sampled 50/50), so the same writer serves read 1 and read 2. The

[1] Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the connector's answer tokens.

untrained condition passes the last visit's full-attention KV and Gated-DeltaNet convolution and recurrent states. Teacher: native 2/2; 20M answer tokens of the same mixture in the SenseNova chat template; peak learning rate 2×10^{-4}.

B Evaluation Details

Base models use lm-evaluation-harness prompts through our runner for GSM8K (3-shot CoT, flexible-extract, 1,319), MMLU (5-shot, 14,042), ARC-Challenge (25-shot, 1,172), BBH (3-shot CoT, 60 per subtask) and MATH-500 (the 500-problem subset of MATH, 4-shot Minerva prompt). RULER (13 tasks $\times$ 40 rows per length) and HELMET (RAG, synthetic recall, re-ranking and ICL categories; 1,108 rows) are prepared and scored by our own scripts. Thinking models answer GSM8K and MATH-500 zero-shot in their chat template, answers in `\boxed{}` scored with math-verify, with budgets of 4,096 and 8,192 tokens; their MMLU-Pro is 5-shot CoT with 30 questions per subject (420) and their BBH 3-shot CoT with 60 per subtask. SenseNova-Loop-9B uses MMLU-Pro with 70 questions per subject (980) through the harness and the full BBH (6,511) through our own scripts. For base models, MATH-500 answers are scored with math-verify on the full generated answer, with and without exemplars. The harness's own math-verify extraction gives -9.8 instead of -11.2 for the 4-shot comparison on Ouro-2.6B; Minerva's strict "Final Answer" match would reverse it (19.6 vs. 9.8) because the read-1 writer drops the required format more often. On raw HELMET prompts thinking checkpoints can emit a newline and stop; for thinking models we therefore mask the stop tokens until an answer has a non-whitespace token, for every configuration. SenseNova RULER uses a non-thinking prefill for every configuration. Native throughput baselines: static per-loop caches with the same two-call flash attention (fastest), the standard dynamic cache (500 / 245 / 105 tokens/s at $d = 1/2/4$), and a preallocated contiguous cache with one attention call (267 / 130 / 57); all produce identical greedy tokens. Nanbeige4.2-3B-Base connectors trained per (p, d) pair, read 2 / write 1 minus read 1 / write 1: RULER 4K/8K/16K $+11.8/+11.7/+14.2$, MMLU $+12.2$, GSM8K $+5.2$. Intervals: paired bootstrap over items, 5,000 resamples; every interval in the paper is recomputed from released per-item scores by one script (Appendix J).

C Training-Seed Robustness

	task	seed 1	seed 2
read 4 − read 1 (write 1)	GSM8K	$+8.0$	$+7.2$
	MMLU	$+15.2$	$+15.2$
	RULER-4K	-3.8	-3.5
	RULER-8K	$+2.5$	$+2.0$
	RULER-16K	$+8.6$	$+8.6$
read 4 − read 2 (write 2)	GSM8K	-1.7	-2.9
	RULER-4K	-7.2	-6.8

Table 3: Ouro-2.6B connector trained with two seeds (initialisation and (p, d) sampling); paired differences within each seed, full sets.

D Read Deep, Write Shallow on Ouro-1.4B-Thinking

Table 4 repeats the comparison of Figure 4 on the smaller Thinking checkpoint. The effect of reading depth replicates: with the same connector, reading four loops instead of two raises MMLU-Pro ($+8.6$), HELMET ($+7.3$), ARC-Challenge ($+2.7$) and BBH ($+8.8$) and shows no detectable difference on GSM8K, MATH-500, HumanEval and MBPP+. Against native 2/2 the result is weaker than on Ouro-2.6B-Thinking: read 4 / write 2 is significantly better on GSM8K ($+3.0$) and ARC-Challenge ($+4.3$), within noise on five benchmarks, and much worse on 3-shot BBH (-15.9), where this checkpoint's connector degenerates at both read depths (Limitations). Against native 4/4 the gap is larger than on Ouro-2.6B-Thinking, most of all on BBH (-37.2) and HELMET (-18.3).

E End-to-End Time and Decode Throughput

Serving. Storing the prompt once also changes throughput (Figure 5; Table 6 in Appendix E). In a controlled decode benchmark (8K context, 32 repeated decode steps, largest batch that fits on one B300), one prompt KV cache decodes $0.97\times$ as fast as the fastest native implementation we measured at one writing loop and 1.19–$1.62\times$ at two to four, because it admits larger batches as d grows; this native baseline uses the same flash attention and is faster than the standard dynamic cache (Appendix B). With long answers (1K-token prompts, 2,048 generated tokens) one prompt KV cache decodes $0.97\times$ native at d=2 and $1.16\times$ at d=4.

	score				read 4 / write 2 minus		
benchmark	native 2/2	read 2 / write 2	read 4 / write 2	native 4/4	native 2/2	read 2 / write 2	native 4/4
GSM8K	86.4	89.5	89.5	92.3	+3.0 [1.3, 4.8]	+0.0 [−1.5, 1.4]	−2.9 [−4.3, −1.5]
MATH-500	76.4	77.6	75.6	85.2	−0.8 [−4.6, 3.0]	−2.0 [−5.8, 1.8]	−9.6 [−12.8, −6.2]
MMLU-Pro	39.8	34.8	43.3	46.2	+3.6 [−1.0, 8.1]	+8.6 [4.3, 12.9]	−2.9 [−6.7, 1.2]
ARC-C	56.1	57.7	60.4	62.5	+4.3 [2.1, 6.4]	+2.7 [1.0, 4.5]	−2.0 [−3.7, −0.4]
BBH	48.7	24.0	32.8	70.0	−15.9 [−18.9, −13.0]	+8.8 [6.2, 11.5]	−37.2 [−39.9, −34.5]
HumanEval	34.1	33.5	34.1	32.9	+0.0 [−6.1, 6.1]	+0.6 [−5.5, 6.7]	+1.2 [−4.9, 7.3]
MBPP+	70.4	74.3	72.2	75.1	+1.9 [−2.1, 5.8]	−2.1 [−5.8, 1.9]	−2.9 [−6.3, 0.5]
HELMET	45.0	39.2	46.5	64.9	+1.5 [−4.8, 7.8]	+7.3 [2.2, 12.7]	−18.3 [−22.8, −13.6]

Table 4: **Figure 4 on Ouro-1.4B-Thinking.** One connector serves read 2 / write 2 and read 4 / write 2; same benchmarks, protocols and items as Figure 4. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

task / model	config	acc. (%)	time (s)
GSM8K Ouro-2.6B-T	native 2/2	95.3	70.0
	native 4/4	93.8	114.2
	read 2 / write 2	96.9	62.6
	read 4 / write 2	96.9	63.1
MATH-500 Ouro-2.6B-T	native 2/2	85.0	243
	native 4/4	82.5	469
	read 4 / write 2	85.0	298
GSM8K SenseNova	native 2/2	98.4	48.0
	read 1 / write 1	96.9	28.4
	read 2 / write 1	95.3	30.6

Table 5: **End to end per answer**, prefill included (batch 1, one B300; Ouro-2.6B-Thinking and SenseNova-Loop-9B; 64 GSM8K and 40 MATH-500 problems, budgets 4,096 and 8,192 tokens). On GSM8K, read 4 / write 2 spends 1.27× the decode time per generated token of native 2/2 at batch 1, but its answers are shorter.

8K context, 32 steps	d=1	d=2	d=3	d=4
native d/d	834	342	200	125
read 4 / write d, one prompt cache	813	406	271	203
1K context, 2,048 steps		d=2		d=4
native d/d		586		219
read 4 / write d, one prompt cache		570		253

Table 6: **Decode throughput** of Ouro-2.6B (tokens/s, best over tested batch sizes, one B300): native model against the connector reading 4 loops with one stored prompt KV cache.

F Design Variants

Table 7 compares design variants. Training the same connector with its maps frozen at the identity, so that a LoRA writer reads the last read loop's KV directly, gives small estimated losses on GSM8K and MMLU, with intervals including zero, but lowers RULER-16K by 11.4 points at read 4 / write 1 and by 2.6–3.4 points at two writing loops. Restricting the key map to a RoPE-commuting scale, which is what allows one stored prompt KV cache, shows no measurable cost against unrestricted key maps with one cache per loop (RULER-16K, five (p, d) pairs).

variant	p/d	task	difference
maps frozen at identity†	4/1	RULER-16K	−11.4 [−14.7, −8.2]
	4/1	GSM8K	−1.7 [−3.6, 0.4]
	4/1	MMLU	−0.2 [−0.6, 0.1]
	4/2	RULER-16K	−2.6 [−4.5, −0.7]
	4/2	GSM8K	−0.2 [−1.5, 1.1]
	2/2	RULER-16K	−3.4 [−5.6, −1.3]
unrestricted keys‡	–	RULER-16K	all CIs include 0

Table 7: **Design variants** on Ouro-2.6B: variant minus our connector at read p / write d (full sets, paired). Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals. †Same data, budget, seed and (p, d) sampling; only the maps are not trained (LoRA only, the writer reads the last read loop's KV directly). ‡Unrestricted key maps with one prompt KV cache per writing loop, at read 1 / write 1, 2/2, 3/1, 3/2 and 4/2.

G SenseNova-Loop-9B Without Training

SenseNova-Loop-9B's one-loop writer collapses on its own. Handing that unchanged writer the prompt state of a two-visit prefill—attention KV and the recurrent state of its linear-attention layers, no parameter changed—lifts RULER-4K from 21.6 to 83.5 and MMLU-Pro from 18.9 to 55.1, whereas GSM8K accuracy only rises from 2.2 to 10.9, far below the 95.8 of native 2/2: deeper reading recovers knowledge and long-context accuracy, not multi-step reasoning.

H Where Reading Deep Does Not Help

HELMET shows where reading deep helps and where it hurts (Table 8). On both Ouro reason-

ing models, reading four loops instead of one improves retrieval-augmented QA by 15 points and re-ranking by 13–18, but lowers exact recall from a JSON key–value store on Ouro-1.4B-Thinking (-23.5); against native 4/4, read 4 / write 2 gains on retrieval-augmented QA and in-context learning ($+6.1$, $+6.6$) and loses exact recall (58.8 vs. 100), so its HELMET average is lower (-8.7 [-13.3, -4.1]). SenseNova-Loop-9B does not follow the Ouro pattern on recall and re-ranking. The recall category is small (34 items on Ouro).

comparison	RAG	re-rank	ICL	recall
2.6B-T, read 4 − 1	+15.4	+13.4	+5.1	−2.9
1.4B-T, read 4 − 1	+14.5	+18.0	+9.3	−23.5
SenseNova, read 2 − 1	+1.2	−32.9	+21.6	+24.0
2.6B-T, 4/2 − native 4/4	+6.1	−6.3	+6.6	−41.2

Table 8: **HELMET by category** (score differences on a 0–100 scale; reading comparisons at one writing loop, same connector). Reading deep helps retrieval-augmented QA (RAG), re-ranking and in-context learning (ICL) on Ouro but can lose exact key–value recall. Items: 700 / 41 / 333 / 34 per category on Ouro, 4,200 / 246 / 2,000 / 200 on SenseNova; intervals are in `paper_numbers.json`.

Reading deeper does not help everywhere (Table 9). At one writing loop it gives a negative but inconclusive estimate on RULER-4K, where answers are copied verbatim, and clearly lowers 4-shot MATH-500 on Ouro-2.6B; without exemplars the same comparison turns positive on two model sizes and two writing depths. Removing exemplars also changes length and format, so we read this as a prompting interaction rather than as evidence that literal information is lost; linear probes of the cache, which find the next token more and (in Ouro) the previous token less linearly decodable after more loops, are consistent with it (Appendix I).

Other design variants. Sharing generated-token KV across writing loops ("shared answer KV", Appendix A) shows no significant difference at two and three writing loops and costs 2.6 GSM8K points at four. A gated branch over the first prefill loop's KV improves RULER-4K by 2.3 points at read 4 / write 1, leaves GSM8K unchanged within noise, and does not remove the 4-shot MATH deficit. (Ouro-2.6B, full sets: shared answer KV at 4/2, 3/3, 4/4: $+0.3$ [-1.5, 2.1], -0.8 [-2.4, 0.8], -2.6 [-4.2, -1.1]; first-loop branch at 4/1: RULER-4K $+2.3$ [1.0, 3.6], GSM8K $+0.7$ [-0.8, 2.2].)

model	write d	task	read 4 − read d
Ouro-2.6B	1	RULER-4K	-3.8 [-8.2, 0.3]
	1	MATH, 4-shot	-11.2 [-15.4, -6.8]
	1	MATH, 0-shot	$+7.0$ [3.0, 10.8]
	2	MATH, 0-shot	$+7.0$ [3.0, 11.2]
Ouro-1.4B	1	MATH, 4-shot	-4.0 [-8.2, 0.0]
	1	MATH, 0-shot	$+4.8$ [1.4, 8.4]

Table 9: **Reading deep depends on context length and prompting.** Same connector, base models, read 4 / write d minus read d / write d; MATH-500 scored with math-verify on the full answer. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

I Additional Analyses

Unfinished answers of a one-loop writer. A reasoning model written with one loop often gives no answer. On Ouro-2.6B-Thinking, native 1/1 gives no boxed answer on 43.1% of GSM8K problems; 43.5% of its answers approach the 4,096-token budget. Training the connector at read 1 / write 1 reduces answers without a boxed answer to 19.5%; with the same writer, reading four loops reduces them further to 12.7% overall, lower in every difficulty bucket (Figure 8, left), and shortens answers by 20% overall. A lexical hesitation-marker rate does not fall (12.2 vs. 13.8 per 1,000 tokens), so shorter answers do not come from fewer markers of hesitation; these counts do not identify the underlying reasoning process. The same writer on SenseNova-Loop-9B, handed the two-visit prompt state without training, leaves no boxed answer on 43.7% of GSM8K problems instead of 96.8% (Figure 8, right).

What each loop stores. What changes in the cache from loop to loop? We fit linear probes to the frozen native model's per-loop value cache—what attention reads back from the prompt—at the same layer and with the same weights, varying only the loop (Figure 9). From loop 1 to loop 4 of Ouro-2.6B the previous token becomes less linearly decodable (57.7% → 45.5%, lower in 12 of 13 probed layers) and the next token more (26.0% → 36.1%, higher in all 13); Ouro-1.4B and Ouro-2.6B-Thinking agree. In SenseNova-Loop-9B the next-token gain replicates in all six attention layers (24.1% → 29.1%) while the previous token is about as decodable at both visits (56.8% vs. 56.5%). These are decodability results, not

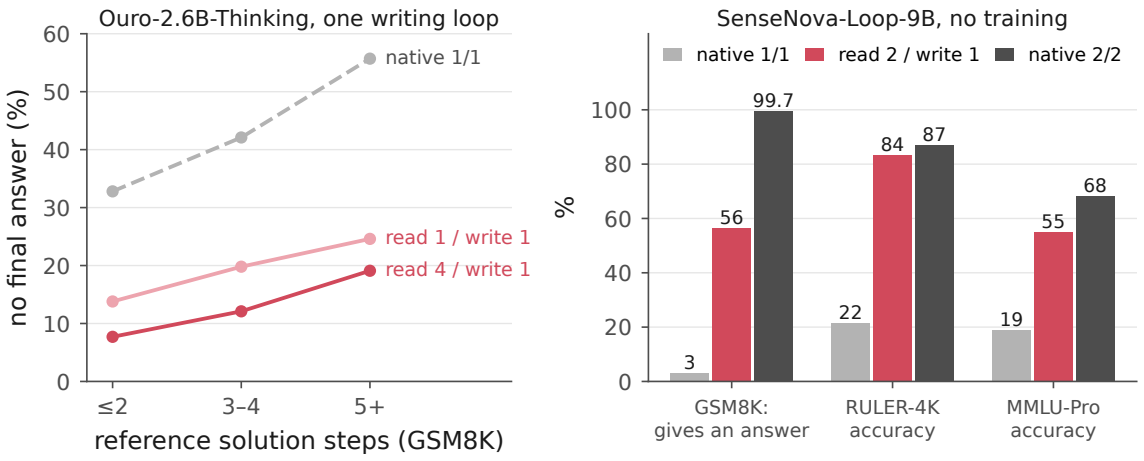


Figure 8: **Deep reading helps a shallow writer.** Left: Ouro-2.6B-Thinking at one writing loop, GSM8K problems bucketed by the number of lines in the reference solution (326 / 668 / 325 problems). Answers without a boxed answer grow with difficulty; reading four loops instead of one lowers them by 5.5–7.7 points in every bucket. Right: SenseNova-Loop-9B without any training: the native one-loop writer reads the prompt state of a two-visit prefill.

proof of information loss or causal use (Merin, 2026), and the probes read the native model, not the adapted writer; they resemble the across-layer trend of Voita et al. (2019), and per-loop probes of looped transformers go back to Yang et al. (2023).

J Reproducibility

`python ouro/scripts/paper_numbers.py compute` recomputes the paired differences, intervals, means, timing ratios and Section 4.3 statistics of this paper from the accompanying per-item records (1 MB) in seconds on a laptop, and compares them with expected values hard-coded in the script, which mirror the numbers printed here. `REPRODUCE.md` maps the remaining results (throughput, probes) to their scripts and gives the training and evaluation commands that regenerate the per-item records.

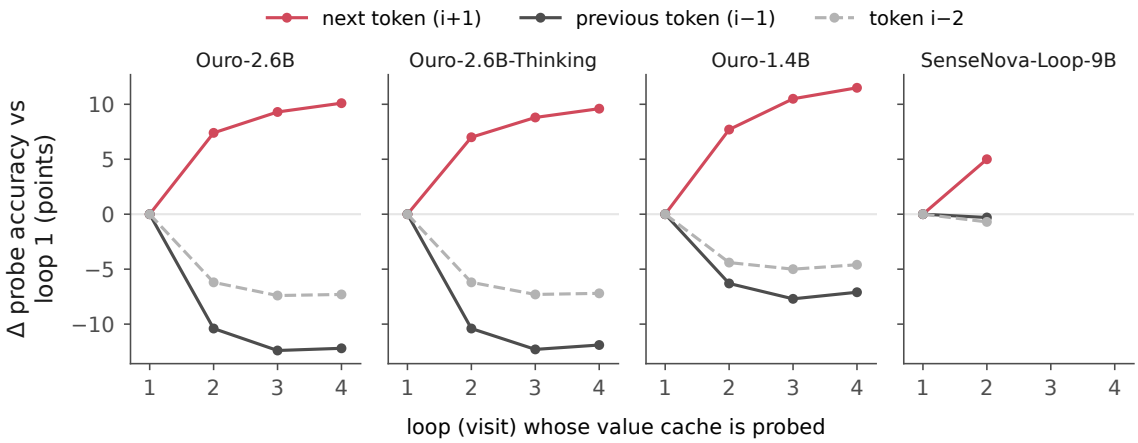


Figure 9: **Later loops make the next token more decodable; in Ouro the previous token becomes less decodable.** Linear probes on the stored value cache, same weights and layer, change from loop 1 (mean over probed layers; 384 FineWeb windows of 1,024 tokens, 20% held out; 2,048 most frequent training tokens).